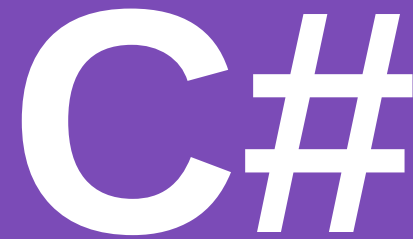


C# tanulási út

Az alapoktól az osztályokig – fájlkezeléssel

Magyar nyelvű jegyzet kezdőknek

Közérthető magyarázatok • működő példakódok • gyakorló feladatok

The C# logo is displayed in white text. It consists of a large 'C' followed by a '#' symbol. The logo is positioned on the right side of the slide, overlaid on a large purple circle. A smaller, darker purple circle is partially visible behind it.

1

Alapok

Első program,
kiírás, beolvasás

2

Változók és típusok

int, string,
konverzió, szöveg

3

Döntések és ciklusok

if, switch, for, while

4

Tömbök és listák

Tömb, List, foreach

5

Metódusok, hibakezelés

Függvények,
try/catch

6

Fájlkezelés

Olvasás, írás,
mentés

7

Osztályok

Objektumok,
öröklődés

Hogyan tanulj ebből? Minden példát gépelj be kézzel Visual Studióban, futtasd, aztán változtass rajta valamit. A diák végén lévő **mini-feladatokat** próbáld meg segítség nélkül megoldani.

- A **C#** (ejtsd: szí-sárp) a Microsoft programozási nyelve. Készíthetsz vele játékot (Unity), weboldalt, asztali programot, mobilappot.
- Amit leírsz, azt a **fordító** (compiler) alakítja géppel érthető programmá. Ez a **.NET** platformon fut.
- Eszköz: **Visual Studio Community** (ingyenes). Ez az írószerkesztő, fordító és hibakereső egyben.
- A munka menete: **kód írása** → **Build (Ctrl+Shift+B)** → **futtatás (F5)**.

Tipp: a **Ctrl+F5** debug nélkül futtat, és a konzolablak a végén nyitva marad, így el tudod olvasni a kimenetet.

```
projekt.sln          <- megoldás (csak egy "tartalomjegyzék")
└─ projekt/
   └─ projekt.csproj  <- a projekt beállításai (.NET verzió)
   └─ Program.cs      <- ITT írod a kódot
   └─ bin/Debug/...    <- az elkészült program (.exe)
   └─ obj/             <- ideiglenes fájlok
```

- A **.sln** (solution = megoldás) nem tartalmaz kódot, csak azt tudja, mely projektek tartoznak össze. Duplakattal ezt nyisd meg.
- A **.csproj** a projekt „személyije”: milyen .NET verziót használ, milyen beállításokkal.
- A **Program.cs** a fő kódfájl. Az **F5** a **bin/Debug** mappába építi a programot. Ide kerülnek a relatív útvonalú fájlok is (lásd: fájlkezelés).

```
using System;

namespace projeckt
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Helló, világ!");
        }
    }
}
```

- **using** – eszköztár behúzása (itt: a Console miatt)
- **namespace** – a kódod „mappája”, a projekt neve
- **class Program** – az osztály, amiben a program él
- **Main** – **innen indul** a program, itt kezdődik a futás
- Minden utasítás végére **;** kell, a **{ }** pedig összetartozó blokkot jelöl

Újabb .NET-ben (6+) a sablon rövidebb is lehet: elég egy sor, `Console.WriteLine("Helló!");`. Ugyanazt csinálja.

```
Console.WriteLine("Ez új sorba ír");  
Console.Write("Ez ugyanabba a sorba: ");  
  
Console.Write("Mi a neved? ");  
string nev = Console.ReadLine() ?? "";  
  
Console.WriteLine("Szia, " + nev + "!");  
Console.WriteLine($"Szia, {nev}!"); // ugyanaz, szebben
```

- `WriteLine` kiír és sort emel, a `Write` nem.
- `ReadLine` beolvassa, amit a felhasználó begépel, **mindig szöveggént**.
- A `$"...{nev}..."` az **interpolált szöveg**: a `{ }` közé változót írhatsz.

A `?? ""` csak a sárga figyelmeztetést (CS8600) kerüli el arra az esetre, ha a beolvasás üres lenne. Elhagyhatod, nem hiba.

```
int kor = 17;           // egész szám
double magassag = 1.82; // tizedes tört
string nev = "Anna";   // szöveg
char betu = 'A';        // egyetlen karakter
bool diak = true;       // igaz / hamis
var szam = 10;           // a C# kitalálja: int
const double PI = 3.14;  // állandó, nem változtatható

kor = 18;               // új érték
kor = kor + 1;          // most 19
kor++;                  // most 20
```

- A változó = **név + típus + érték**.
- Egy változót **egyszer** deklarálsz (típussal), utána csak értéket adsz neki.
- Névadás: kisbetűvel kezd, ne kezdődjön számmal, ne legyen benne szóköz. Pl. **maxPont**.
- A tizedes **a kódban mindig pont**: **1.82**.

A leggyakoribb adattípusok

Típus	Mit tárol?	Példa
<code>int</code>	egész szám	42, -7
<code>double</code>	tizedes tört	3.14
<code>decimal</code>	pontos tizedes (pénz)	19.99m
<code>string</code>	szöveg	"Anna"
<code>char</code>	egyetlen karakter	'A'
<code>bool</code>	igaz vagy hamis	true, false

Szabály: a típus határozza meg, mit tehetsz az értékkel. `int`-ből nem lesz szöveg magától, és `string`-gel nem tudsz számolni. Erre jó a konverzió (következő dia).

```
Console.Write("Hány éves vagy? ");
string szoveg = Console.ReadLine() ?? "";

// 1) Egyszerű, de elszáll, ha nem szám:
int kor = int.Parse(szoveg);

// 2) Biztonságos:
if (int.TryParse(szoveg, out int jo))
    Console.WriteLine("Jövőre " + (jo + 1) + " leszel.");
else
    Console.WriteLine("Ez nem szám!");

double d = 3.99;
int egesz = (int)d;           // 3 (levágja a tizedest)
string s = 42.ToString();    // "42"
```

- A `ReadLine` mindig **szöveget** ad, ezt kell számmá alakítani.
- `int.Parse` hibás bemenetnél **kivételt dob** (a program összeomlik).
- `int.TryParse` nem omlik össze: igaz/hamis értéket ad vissza.

Magyar gépen: `double.Parse` beolvasásnál a gép nyelvi beállítása számít. Ott a tizedesjel a **vessző** (1,5), a pont (1.5) hibát okozhat.

Fajta	Operátorok	Példa
Számolás	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	<code>7 % 2 → 1</code> (maradék)
Összehasonlítás	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code>	<code>kor >= 18</code>
Logikai	<code>&&</code> (és) <code> </code> (vagy) <code>!</code> (nem)	<code>kor >= 18 && diak</code>
Értékadás	<code>=</code> <code>+=</code> <code>-=</code> <code>++</code> <code>--</code>	<code>pont += 5;</code>

Gyakori csapda: egész számok osztása egészet ad. `7 / 2` az `3`, nem `3,5`! Ha tizedes eredmény kell, írd így: `7 / 2.0`. Az **egyenlőség vizsgálata** `==`, az `=` értéket ad.

```
string s = "Hello Világ";

Console.WriteLine(s.Length);           // 11
Console.WriteLine(s.ToUpper());        // HELLO VILÁG
Console.WriteLine(s.ToLower());        // hello világ
Console.WriteLine(s.Contains("Vil")); // True
Console.WriteLine(s.Substring(0, 5));  // Hello
Console.WriteLine(s.Replace("l", "L"));
Console.WriteLine(s.Trim());           // szélső szóközök le

string[] szavak = "alma,körte,szilva".Split(',');
Console.WriteLine(szavak[1]);          // körte
```

- A szövegen sok **beépített művelet** van, ponttal hívod őket.
- A karakterek számozása **0-tól** indul.
- A **Split** szétvágja a szöveget, ez a fájlkezelésnél nagyon jól jön majd.

```
int pont = 78;

if (pont >= 90)
    Console.WriteLine("Jeles");
else if (pont >= 75)
    Console.WriteLine("Jó");
else if (pont >= 50)
    Console.WriteLine("Elégséges");
else
    Console.WriteLine("Elégtelen");

int kor = 20;
bool diak = true;
if (kor >= 18 && diak)
{
    Console.WriteLine("Felnőtt diák");
    Console.WriteLine("Több sor = kell a { }");
}
```

- A program **fentről lefelé** vizsgálja a feltételeket, és az **első igaz** ágat hajtja végre.
- Egy utasításnál elhagyható a **{ }**, többnél **kötelező**.
- A feltétel **bool** (igaz/hamis) értékű kifejezés legyen.

```
Console.Write("A hét napja (1-7): ");
int nap = int.Parse(Console.ReadLine() ?? "1");

switch (nap)
{
    case 1:
        Console.WriteLine("Hétfő");
        break;
    case 2:
        Console.WriteLine("Kedd");
        break;
    case 6:
    case 7:
        Console.WriteLine("Hétfő");
        break;
    default:
        Console.WriteLine("Hétköznapi");
        break;
}
```

- A **switch** akkor jó, ha **egy érték** sok lehetséges esetét kell szétválogatni.
- Minden **case** végén kell a **break**.
- A **default** az „egyik sem” ág.
- Több **case** egymás alatt = ugyanaz az ág.

```
// for: ha tudod, hányszor
for (int i = 1; i <= 5; i++)
    Console.WriteLine(i);

// while: amíg a feltétel igaz
int n = 3;
while (n > 0)
{
    Console.WriteLine(n);
    n--;
}

// foreach: egy gyűjtemény minden elemén
foreach (char betu in "abc")
    Console.WriteLine(betu);
```

- **for**: számlálós, előre ismert ismétlésszám.
- **while**: addig megy, amíg a feltétel igaz. Ha sosem lesz hamis, **végtelen ciklus** lesz!
- **foreach**: végigmegy egy gyűjtemény elemein (tömb, lista, szöveg).

Mini-feladat: Írj programot, ami kiírja a szorzótáblát 1-től 10-ig **for** ciklussal!

```
// Tömb: fix méretű
int[] jegyek = { 5, 4, 3, 5 };
Console.WriteLine(jegyek[0]);    // 5 (0-tól indexelünk!)
Console.WriteLine(jegyek.Length); // 4

// List: bármikor nő vagy csökken
List<string> nevek = new List<string>();
nevek.Add("Anna");
nevek.Add("Bence");
nevek.Add("Cili");
nevek.Remove("Anna");
Console.WriteLine(nevek.Count);   // 2

foreach (string n in nevek)
    Console.WriteLine(n);
```

- **Tömb**: fix darabszámú, azonos típusú elemek.
- **List<T>**: rugalmas lista, a **T** helyére az elem típusa kerül.
- Az index **0-tól** indul, az utolsó elem indexe **Count - 1**.
- Hibás index (**jegyek[10]**) → **IndexOutOfRangeException**.

A **List** használatához kellhet a **using System.Collections.Generic;** sor. Újabb projektekben ez már automatikus.

```
Random rnd = new Random();
int titok = rnd.Next(1, 101); // 1 és 100 között
int tipp = 0;
int probak = 0;

while (tipp != titok)
{
    Console.Write("Tipp (1-100): ");
    int.TryParse(Console.ReadLine(), out tipp);
    probak++;

    if (tipp < titok)
        Console.WriteLine("Több!");
    else if (tipp > titok)
        Console.WriteLine("Kevesebb!");
}
Console.WriteLine($"Eltaláltad {probak} tippből!");
```

- Ebben **minden eddigi dolog** benne van: változó, ciklus, feltétel, konverzió.
- **Random** = véletlenszám-generátor.
- Ha a játékos nem számot ír, a **TryParse** 0-t ad, így nem omlik össze.

Bővítés: számolj tippelési limitet, vagy kérdezd meg a végén, akar-e új játékot!

```
static int Osszead(int a, int b)
{
    return a + b;
}

static void Koszon(string nev)
{
    Console.WriteLine($"Szia, {nev}!");
}

static bool Paros(int szam)
{
    return szam % 2 == 0;
}

// A Main-ben:
int eredmeny = Osszead(3, 4);    // 7
Koszon("Anna");
Console.WriteLine(Paros(10));    // True
```

- Felépítés: **visszatérési típus + név + (paraméterek) + törzs.**
- A **return** ad vissza értéket, a **void** azt jelenti: **nem ad vissza semmit.**
- Egy metódus = egy feladat. Így a kód olvasható marad, és nem kell ismételtetni.
- A metódusokat a **Program** osztályba írd, a **Main** mellé.

```
try
{
    Console.Write("Szám: ");
    int x = int.Parse(Console.ReadLine() ?? "");
    Console.WriteLine(100 / x);
}
catch (FormatException)
{
    Console.WriteLine("Ez nem szám!");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Nullával nem osztunk!");
}
catch (Exception ex)
{
    Console.WriteLine("Egyéb hiba: " + ex.Message);
}
finally
{
    Console.WriteLine("Ez mindig lefut.");
}
```

- A **try** blokkba az a kód kerül, ami **hibázhat**.
- Ha hiba (**kivétel**) történik, a program a megfelelő **catch** ágra ugrik, nem omlik össze.
- A **finally** mindig lefut, hiba esetén is.
- Sorrend: előbb a **konkrét**, a végén az általános **Exception**.

A fájlkezelésnél ez létfontosságú: fájl bármikor hiányozhat vagy zárolt lehet.

A program memóriája **törlődik**, amikor bezárod. Ha adatot meg akarsz tartani (mentés, napló, beállítás), **fájlba** kell írnod. Kell hozzá: `using System.IO;`

Metódus	Mit csinál?
<code>File.WriteAllText(út, szöveg)</code>	Fájlba ír (új fájlt hoz létre, vagy felülírja a régit)
<code>File.ReadAllText(út)</code>	Az egész fájlt egy szöveggént beolvassa
<code>File.AppendAllText(út, szöveg)</code>	A fájl végére fűz , a régi tartalom megmarad
<code>File.WriteAllLines(út, sorok)</code>	Egy tömb/lista elemeit sorokként írja ki
<code>File.ReadAllLines(út)</code>	Soronként beolvassa, <code>string[]</code> -t ad
<code>File.Exists(út) / File.Delete(út)</code>	Létezik-e a fájl? / Törlés

```
using System.IO;

// Írás (új fájl, vagy felülírás)
File.WriteAllText("adat.txt", "Első sor");

// Olvasás
string tartalom = File.ReadAllText("adat.txt");
Console.WriteLine(tartalom);           // Első sor

// Hozzáadás a végéhez
File.AppendAllText("adat.txt", "\nÚj sor");

Console.WriteLine(File.ReadAllText("adat.txt"));
// Első sor
// Új sor
```

- Ez a **legegyszerűbb** módszer, kis fájlokhoz tökéletes.
- A `\n` új sort jelent a szövegben.
- A `"adat.txt"` **relatív útvonal**: a program mappájában keresi/hozza létre (részletek: 23. dia).

Vigyázz: a `WriteAllText` minden hívásnál **felülírja** a fájlt! Hozzáadásához az `AppendAllText` kell.

```
string[] nevek = { "Anna", "Bence", "Cili" };
File.WriteAllLines("nevek.txt", nevek);

string[] beolvasott = File.ReadAllLines("nevek.txt");
foreach (string sor in beolvasott)
    Console.WriteLine(sor);

if (File.Exists("nevek.txt"))
    Console.WriteLine("A fájl létezik.");
else
    Console.WriteLine("Nincs ilyen fájl.");

File.Delete("nevek.txt");    // törlés
```

- Listát is kiírhatsz:
`WriteAllLines("f.txt", lista).`
- A `ReadAllLines` után `foreach`-cel egyenként feldolgozhatod a sorokat.
- Olvasás előtt **mindig** ellenőrizd a `File.Exists`-szel, hogy van-e fájl!

```
using (StreamWriter iro = new StreamWriter("naplo.txt", append: true))
{
    iro.WriteLine("A program elindult: " + DateTime.Now);
    iro.WriteLine("Második bejegyzés");
}

using (StreamReader olvaso = new StreamReader("naplo.txt"))
{
    while (!olvaso.EndOfStream)
    {
        string sor = olvaso.ReadLine() ?? "";
        Console.WriteLine(sor);
    }
}
```

- Nagyobb fájlnál vagy ha **sorról sorra** akarsz dolgozni, ez a módszer.
- A **using** blokk a végén **automatikusan lezárja** a fájlt. Ez nagyon fontos!
- **append: true** = hozzáfűz, **false** (alap) = felülír.

Ha lezáratlanul marad egy fájl, később „**a fájlt másik folyamat használja**” hibát kapsz.

```
// Hol keres a program?  
Console.WriteLine(Directory.GetCurrentDirectory());  
// pl. ...\\projekkt\\bin\\Debug\\net8.0  
  
// Teljes (abszolút) útvonal: @ jel kell a \\ miatt  
string fajl = @"C:\\Users\\Anna\\Documents\\adat.txt";  
  
// Biztonságos összefűzés  
string mappa = @"C:\\Adatok";  
string ut = Path.Combine(mappa, "adat.txt");  
  
// Mappa létrehozása, ha nincs  
if (!Directory.Exists(mappa))  
    Directory.CreateDirectory(mappa);
```

- **Relatív** útvonal ("adat.txt"): a futó program mappájából indul, ez általában a bin\\Debug\\... mappa.
- **Abszolút** útvonal: a teljes cím a meghajtótól.
- A \\ speciális karakter a szövegben, ezért írd elé @-ot, vagy duplázd meg: "C:\\\\Adatok".
- A **Path.Combine** helyesen teszi össze az útvonalat.

Hibaüzenet / tünet	Miért van?	Javítás
<code>FileNotFoundException</code>	A fájl nincs ott, ahol keresed (rossz név vagy mappa)	<code>File.Exists</code> ellenőrzés; írd ki a <code>GetCurrentDirectory()</code> -t
<code>DirectoryNotFoundException</code>	A mappa nem létezik	<code>Directory.CreateDirectory(...)</code> előbb
<code>UnauthorizedAccessException</code>	Nincs jogod oda írni (pl. <code>C:\</code> gyökér)	Írj a Documents vagy a program mappájába
<code>IOException</code> (használatban)	Másik program vagy lezáratlan stream fogja	<code>using</code> blokk; zárd be a fájlt Jegyzettömbben
Elveszett az adat	<code>WriteAllText</code> felülírt	<code>AppendAllText</code> vagy <code>append: true</code>
Ékezetek elromlanak (ő, ú)	Rossz kódolás	<code>File.WriteAllText(út, szöveg, Encoding.UTF8); using System.Text;</code>

```
string fajl = "jegyzetek.txt";

while (true)
{
    Console.WriteLine("1 - Új jegyzet | 2 - Lista | 0 - Kilépés");
    string valasz = Console.ReadLine() ?? "";

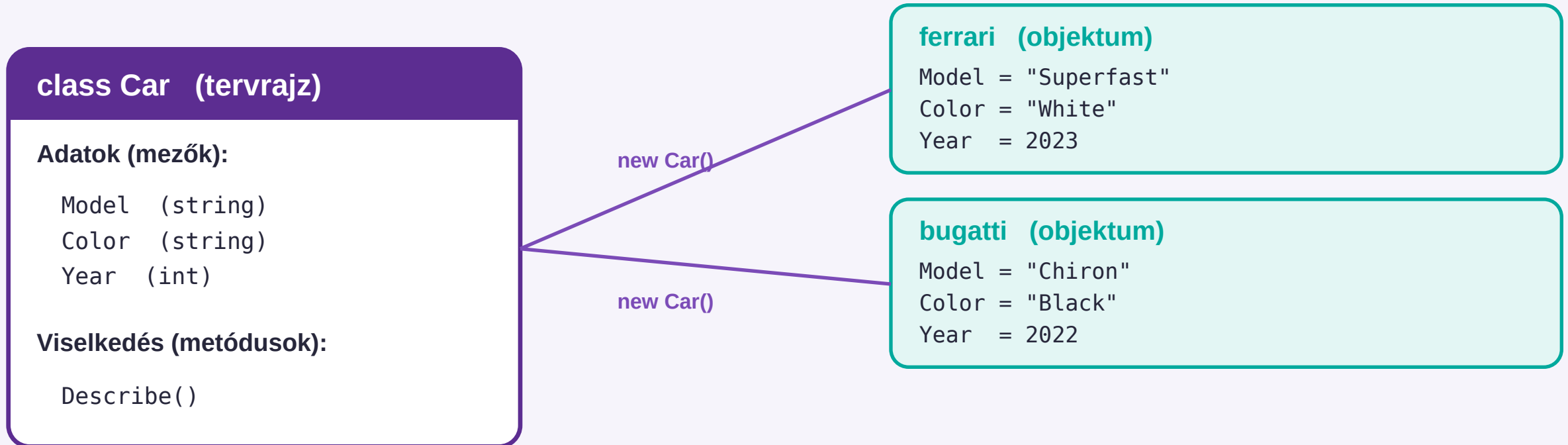
    if (valasz == "1")
    {
        Console.Write("Jegyzet: ");
        string uj = Console.ReadLine() ?? "";
        File.AppendAllText(fajl, uj + Environment.NewLine);
    }
    else if (valasz == "2")
    {
        if (File.Exists(fajl))
            Console.WriteLine(File.ReadAllText(fajl));
        else
            Console.WriteLine("Még nincs jegyzet.");
    }
    else if (valasz == "0")
    {
        break;
    }
}
```

- Összekapcsolja a **ciklust**, **feltételeket** és a **fájlkezelést**.
- A **while (true)** végtelen ciklus, a **break** lép ki belőle.
- A jegyzetek a **program bezárása után is megmaradnak**.

Bővítés: jegyzet törlése, számozott lista (**ReadAllLines** + **for**), keresés a jegyzetek közt.

Osztályok: mire jók, és hogyan működnek?

7 · OSZTÁLYOK



- Az **osztály** egy **tervrajz**: leírja, milyen adatai és tudásai lesznek valaminek.
- Az **objektum** a tervrajzból gyártott **konkrét példány**. Belőle annyit csinálhatsz, amennyit akarsz.

Osztályok nélkül minden autóhoz külön változó kellene (`ferrariModel`, `bugattiModel`...). Osztállyal minden **egy csomagban** van.

```
internal class Program
{
    static void Main(string[] args)
    {
        Car ferrari = new Car();
        ferrari.Model = "Superfast";
        ferrari.Color = "White";
        ferrari.Year = 2023;

        Car bugatti = new Car();
        bugatti.Model = "Chiron";
        bugatti.Color = "Black";
        bugatti.Year = 2022;

        Console.WriteLine(ferrari.Model);
        Console.WriteLine(bugatti.Model);
    }
}

public class Car
{
    public string Model;
    public string Color;
    public int Year;
}
```

- A `Car` és a `Program` két külön osztály. Mindkettő saját `{ }` blokkban van.
- A `new Car()` új objektumot hoz létre.
- A `public` kell, hogy a `Program` is lássa a mezőket.
- Az objektum adataihoz **ponttal** férsz hozzá: `ferrari.Model`.

Gyakori hiba: ha a `Main` véletlenül a `Car` kapcsos zárójelén belül marad, akkor is lefut, de rossz szerkezet. Mindig számold meg, hogy minden `{`-nek van-e `}` párja!

```
public class Car
{
    public string Model { get; set; }
    public string Color { get; set; }
    public int Year { get; set; }

    // Konstruktor: a "new Car(...)" ezt futtatja
    public Car(string model, string color, int year)
    {
        Model = model;
        Color = color;
        Year = year;
    }
}

// Használat:
Car ferrari = new Car("Superfast", "White", 2023);
Car bugatti = new Car("Chiron", "Black", 2022);
Console.WriteLine(ferrari.Model);
```

- **Konstruktor:** egy különleges metódus, **az osztály nevét viseli**, és létrehozáskor fut le. Így nem maradhat üresen egy adat.
- **Tulajdonság** (`{ get; set; }`): a mező „szebb” változata. Ez a szokásos C# módszer. Nagy kezdőbetűvel írjuk.

Ha írsz konstruktort, a `new Car()` (paraméter nélkül) már **nem működik**, mindig meg kell adni az adatokat.

```
public class Car
{
    public string Model { get; set; }
    public int Year { get; set; }
    public int Speed { get; private set; }

    public Car(string model, int year)
    {
        Model = model;
        Year = year;
    }

    public void Gyorsit(int mennyit)
    {
        Speed += mennyit;
    }

    public override string ToString()
    {
        return $"{Year} {Model} ({Speed} km/h)";
    }
}
```

- Az osztály nemcsak **adatot**, hanem **viselkedést** is tárol: `a.Gyorsit(100)`.
- A metódus az objektum **saját adataival** dolgozik.
- A `ToString` felülírásával a `Console.WriteLine(a)` szépen kiírja az objektumot.
- `private set`: kívülről csak olvasható, belül módosítható.

Használat: `Car a = new Car("Chiron", 2022);`
majd `a.Gyorsit(100);` és
`Console.WriteLine(a);` → 2022 Chiron (100 km/h)

```
public class Car
{
    private int _year;

    public int Year
    {
        get { return _year; }
        set
        {
            if (value < 1886)
                throw new ArgumentException("Túl korai év!");
            _year = value;
        }
    }
}

// Használat:
Car a = new Car();
a.Year = 2023;    // rendben
a.Year = 1500;    // kivételt dob!
```

- **public**: bárhonnán elérhető. **private**: csak az osztályon belül látszik.
- Az **egységbezárás** (encapsulation) lényege: az osztály maga **őrzi az adatai épségét**.
- A **set** ágban ellenőrizhetjük az új értéket, és ha rossz, hibát dobunk.
- A **value** a beállítani kívánt új érték.

Ökölszabály: a mező legyen **private**, amit kívülről is kell látni, azt tedd **public** tulajdonsággá.

```
public class Car
{
    // Model, Color, Year és a konstruktor: mint eddig

    public virtual void Describe()
    {
        Console.WriteLine($"{Year} {Color} {Model}");
    }
}

public class ElectricCar : Car
{
    public int BatteryKwh { get; set; }

    public ElectricCar(string model, string color, int year, int kwh)
        : base(model, color, year)
    {
        BatteryKwh = kwh;
    }

    public override void Describe()
    {
        base.Describe();
        Console.WriteLine($"    Akku: {BatteryKwh} kWh");
    }
}
```

- `: Car` azt jelenti: az `ElectricCar` egy `Car`, plusz saját extrái.
- A `base(...)` a szülő konstruktorát hívja meg.
- `virtual` a szülőben = „felülírható”, `override` a gyerekekben = „én máshogy csinálom”.
- Így minden típus a saját módján csinálja ugyanazt (**polimorfizmus**).

```
using System.Linq;

List<Car> garazs = new List<Car>
{
    new Car("Superfast", "White", 2023),
    new Car("Chiron", "Black", 2022),
    new ElectricCar("Taycan", "Blue", 2024, 93)
};

foreach (Car a in garazs)
    a.Describe();           // mindegyik a saját módján

var ujak = garazs.Where(a => a.Year >= 2023);
var rendezett = garazs.OrderBy(a => a.Year);
int db = garazs.Count(a => a.Color == "Black");

foreach (Car a in ujak)
    Console.WriteLine(a.Model);
```

- A lista **bármilyen típusú** elemet tárolhat, akár saját osztályt is.
- Egy `List<Car>`-ba az `ElectricCar` is bekerülhet, mert az is `Car`.
- **LINQ**: szűrés, rendezés, számolás egy sorban. A `a => ...` a „lambda”: *, „minden `a`-ra, ami...”*.

Feladat: Írj `Person` osztályt, aminek van egy `List<Car> Autok` tulajdonsága, és `BuyCar(Car)` meg `ListCars()` metódusa!

```
static void Mentos(List<Car> autok, string fajl)
{
    List<string> sorok = new List<string>();
    foreach (Car a in autok)
        sorok.Add($"{a.Model};{a.Color};{a.Year}");

    File.WriteAllLines(fajl, sorok);
}

static List<Car> Betoltes(string fajl)
{
    List<Car> autok = new List<Car>();
    if (!File.Exists(fajl))
        return autok;           // nincs fájl: üres lista

    foreach (string sor in File.ReadAllLines(fajl))
    {
        string[] r = sor.Split(';');
        autok.Add(new Car(r[0], r[1], int.Parse(r[2])));
    }
    return autok;
}
```

- Itt jön össze **minden**: osztály, lista, szöveg, ciklus, metódus és fájlkezelés.
- **Mentés**: minden autót egy sorra alakítunk, az adatokat **;** választja el (ez a **CSV** formátum).
- **Betöltés**: soronként beolvasunk, `Split(';')`-vel szétvágunk, és az adatokból új **Car** objektumot építünk.
- Mindkét metódust a **Program** osztályba írd, a **Main** mellé.

```
static void Main(string[] args)
{
    string fajl = "autok.txt";

    List<Car> garazs = Betoltes(fajl);
    Console.WriteLine($"Betöltve: {garazs.Count} autó");

    garazs.Add(new Car("Superfast", "White", 2023));
    Mentek(garazs, fajl);

    foreach (Car a in garazs)
        Console.WriteLine($"{a.Year} {a.Color} {a.Model}");
}
```

- Minden futtatásnál **betölti** az előzőleg mentett autókat, hozzáad egyet, és **elmenti** az egészet.
- Futtasd többször: a garázs **minden alkalommal nő**. Az adat a program bezárása után is megmarad.
- Nyisd meg az **autok.txt**-t a **bin\Debug\...** mappában, és nézd meg, hogyan néz ki belülről!

Hibátűrőbb verzió: tedd a **Betoltes** hívást **try/catch** blokkba, és a soroknál ellenőrizd az **r.Length == 3** feltételt.

Téma	Feladat
Alapok, változók	Kérd be a nevedet és a születési évedet, írd ki, hány éves vagy.
Döntések	Kérj be egy számot, és írd ki: negatív, nulla vagy pozitív? Páros vagy páratlan?
Ciklusok	Írd ki a 1–100 közötti páros számok összegét. Aztán a FizzBuzz feladat.
Listák	Kérj be számokat 0-ig, tárold listában, írd ki az átlagot, a legnagyobbat.
Metódusok	Írj <code>Faktorialis(int n)</code> metódust. Majd <code>bool Prim(int n)</code> -t.
Fájlkezelés	Mentsd el a legjobb pontszámot a számkitalálós játékhoz fájlba.
Osztályok	Készíts <code>Konyv</code> osztályt (cím, szerző, év), és mentsd a könyvtárat fájlba.
Kihívás	Egy <code>Dolgozo</code> őssztály és <code>Fonok</code> leszármazott, <code>ToString</code> felülírással.

- **Interfészek** (`interface`) és **absztrakt osztályok** (`abstract`): közös „szerződések” osztályoknak.
- **Dictionary<K,V>**, **enum**, **static**, `struct`: további hasznos eszközök.
- **JSON mentés** (`System.Text.Json`): adatok korszerűbb fájlformátumban.
- **Grafikus felület**: WinForms vagy WPF, ha ablakos programot szeretnél.
- **Weboldal** ASP.NET-tel, **játék** Unity-vel: itt mindenhol az osztályok a főszereplők.

Ingyenes forrás: **Microsoft Learn** C# kezdő útvonala (learn.microsoft.com/dotnet/csharp). A legjobb tanulás: építs saját kis projektet!